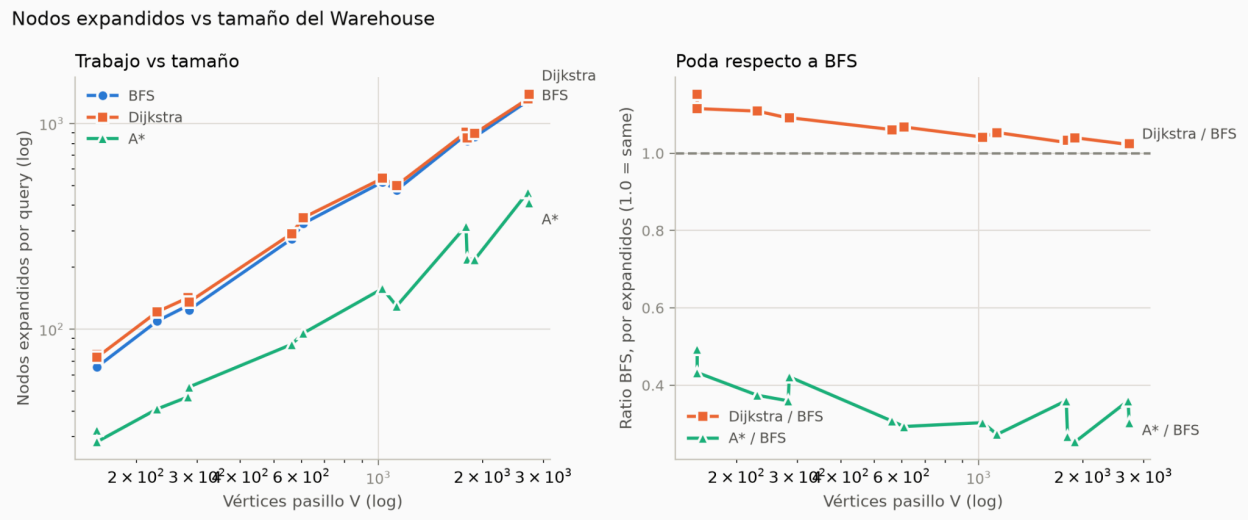
Proyecto Final

Algoritmos y Estructuras de Datos II

Matias Cuneo 15215
Pablo Morata 14763
Emiliano Germani 13804
Mariano Robledo 12985

Sección 21

- Trabajo realizado por algoritmos de búsqueda relacionado con el tamaño del warehouse en una escala log-log:



- wall_ms es la media de tiempo que le tomó a la simulación terminar (1 repeticiones), no solo planear la ruta. % negativo = menos que la primera estrategia.

basico

metric	A - Sin estrategia	B - Centroide de pedidos	C - Clustering espectral	B - Centroide de pedidos vs A (%)	C - Clustering espectral vs A (%)
ticks	122	122	81	0.000	-33.6
assignments	5	5	4	0.000	-20.0
planned_distance	163.0	163.0	125.0	0.000	-23.3
steps	163	163	125	0.000	-23.3
waits	1	1	1	0.000	0.000
reroutes	0	0	0	-	-
side_steps	1	1	1	0.000	0.000
wall_ms	62.9	38.1	112.5	-39.4	78.9

grande

metric	A - Sin estrategia	B - Centroide de pedidos	C - Clustering espectral	B - Centroide de pedidos vs A (%)	C - Clustering espectral vs A (%)
ticks	1,698	1,542	497	-9.187	-70.7
assignments	40	11	9	-72.5	-77.5
planned_distance	6,251.0	4,779.0	1,516.0	-23.5	-75.7
steps	6,257	4,788	1,521	-23.5	-75.7
waits	18	19	8	5.556	-55.6
reroutes	4	6	2	50.0	-50.0
side_steps	0	2	2	-	-
wall_ms	9,073.4	7,720.6	6,443.1	-14.9	-29.0

Sección 22

1. **¿Cómo fue modelado el Warehouse?**

El modelado del warehouse fue mediante LinkedList, Diccionarios, Grafos
La representación de WareHouse está dada como:

```
self.floor: Floor = floor
self.width: int = floor.width
self.height: int = floor.height
self.graph: WeightedDirectedGraph = graph

self.racks: LinkedList = floor.racks
self.doors: LinkedList = floor.doors
self.zones: LinkedList = floor.zones

self.products: CloseDictionary = products
self.pickers: LinkedList = pickers
self.orders: LinkedList = orders
self.orders_by_id: Dictionary = orders_by_id
self.wave_config: WaveConfig = wave_config
self.waves: LinkedList = LinkedList()
self.rejected_orders: list = []
```

El floor es el plan de piso del warehouse, con ubicaciones de cosas fijas con coordenadas <x, y> se incluye el tamaño y la altura que tiene el mismo. Cuenta con el grafo de recorrido de los picker, el mismo es direccionado yo que hay pasillos con sentidos únicos.
Cuenta con las listas de los racks, puertas y de las zonas de despacho. Además tiene un diccionario con todos los productos que pertenecen y con una lista de picker que operan el warehouse. Tiene su lista de pedidos y un diccionario de pedidos para poder encontrarlo más rápido que por una búsqueda en las listas. Una lista de pedidos rechazados

También cuenta con la configuración que tiene que tiene que tener una wave y una lista de las waves
La clase del floor está dada por:

```
self.width: int = width
self.height: int = height

self.grid: CloseDictionary = CloseDictionary(width * height)
self.hall_count: int = 0

self.rack_cells: CloseDictionary = CloseDictionary(width * height)
self.racks: LinkedList = LinkedList()
self.racks_by_id: CloseDictionary = CloseDictionary(2 * rack_count + 1)

self.doors: LinkedList = LinkedList()
self.doors_by_id = CloseDictionary(2 * door_count + 1)
self.zones = LinkedList()
self.zones_by_id = CloseDictionary(2 * zone_count + 1)
self.dropoff: CloseDictionary = CloseDictionary(width * height)

self.sections = LinkedList()
self.sections_by_id = CloseDictionary(2 * section_count + 1)
```

Consta del ancho y alto de la grilla, cuenta con grilla que es un diccionario donde cada celda tiene la información de si mismo (rack, puerta, etc) un contador de pasillos. Contiene un diccionario con celdas de racks que contiene la información de los productos que pertenecen a ese rack y posición de memoria. También tiene la lista de todos los rack un un diccionario los racks por su id para buscarlo más rápido.
Tanto las puertas como las zonas y las secciones tienen la misma lógica que los racks, con lista y diccionarios para búsquedas optimizadas. Por último tiene un diccionario objetos despachados que tiene la información de los objetos despachado
Estas son una de las tantas clases que pertenecen al modelado del warehouse. Esta clase se va armando mediante un paquete de servicios que cuenta con todos los métodos y las verificaciones de integridad del warehouse.

2. **¿Qué representación de la estructura de datos utilizaron y por qué?**

Se usó LinkedList vista en Algo 1

Se aplicó diccionario o hashtable con resolución de colisiones mediante encadenamiento y con una función de hash **djb2,** creado por Daniel J. Bernstein, combinado con el método del módulo para determinar la posición dentro de una tabla hash.
El algoritmo djb2 utiliza:
$h_{i+1} = 33.h_i + ord(c_i)$, con h_0=5381. Luego el índice = h % m

Se implementó la estructura de grafo con listas de adyacencia mediante LinkedList y un diccionario que guarda los vértices

3. **¿Qué algoritmos utilizaron para resolver los problemas encontrados?**
   Algoritmos implementados:
   1. *BFS*
   2. *Dijkstra*
   3. *A**
   4. *Spectral Clustering con K-Mean*
   5. *Curva de Hilbert*

4.

Curva de Hilbert:

Ante la intención de clusterizar el warehouse para detectar zonas de mayor concentración de productos y así optimizar las distancias recorridas por los pickers se exploró la implementación del algoritmo de Curva de Hilbert, las ventajas del mismo es su dependencia exclusiva de métodos matemáticos no iterativos, logrando una complejidad algorítmica de O(n log n) para la generación de k zonas dentro del cluster.

Esta implementación se dejó de lado por Spectral Clustering debido a la inexactitud del mismo al no considerar las distancias topológicas de los puntos, generando imprecisiones al momento de dividir los productos.

Se podría llegar a considerar su uso en caso de que una prioridad sea la reducción de la complejidad algorítmica y de cálculo del sistema.

Spectral Clustering con K-Mean:

La elección definitiva de este estuvo determinada por la precisión, adaptabilidad y el uso de métodos algebraicos en su funcionamiento lo que lo hace determinista*.

Es capaz de detectar y separar zonas complejas que a partir de métodos más simples como K-Mean puro o Curva de Hilbert no sería posible.

Se optó por una implementación determinista de K-Mean determinando la posición inicial de los centroides de manera arbitraria. Esto afecta a la velocidad de conversión pero manteniéndola dentro del límite de iteraciones determinadas.

* La sección algebraica del clustering espectral es determinista pero la implementación de K-Mean no lo es completamente, esta realiza un procedimiento iterativo de corrección del error cuadrático pero limitado a una cantidad de iteraciones fija (100).

Algoritmos de Búsqueda

Como los tres algoritmos encuentran siempre el mismo recorrido (comportamiento esperado), se eligió un algoritmo correspondiente en base a la necesidad de uso (tabla de complejidades medidas en Fig T2).

escenario	V	E	algoritmo	queries	time_ms	expandidos	examinados
basico	230	692	BFS	40	1.362	109.5	336.9
basico	230	692	Dijkstra	40	2.173	121.4	371.6
basico	230	692	A*	40	0.923	40.9	121.7
grande	1,898	5,963	BFS	40	11.1	860.5	2,736.9
grande	1,898	5,963	Dijkstra	40	17.7	894.9	2,844.4
grande	1,898	5,963	A*	40	5.730	217.9	672.0

Figura T2: cada query significa un par de pasillos alcanzables (src, dst); time es la media de 5 repeticiones. expandidos = nodos procesados, examinados = aristas observadas.

5. **¿Qué ocurre si todos los pasillos tienen el mismo largo?**

El modelado de los pasillos es independiente al largo de los mismos ya que la unidad mínima es el nodo de la grilla correspondiente a un pasillo y se une a todos los que comparten esa característica
Si un picker pasa por uno de estos nodos se considera 1 unidad recorrida.

6. **¿Cómo cambia el comportamiento del algoritmo cuando aumenta el tamaño del Warehouse?**

Los algoritmos de búsqueda implementados para las distintas funciones involucradas en la simulación del warehouse escalan de manera proporcional y lineal a medida que aumenta el tamaño del warehouse (visualización de nuestra medición en la Fig C1).

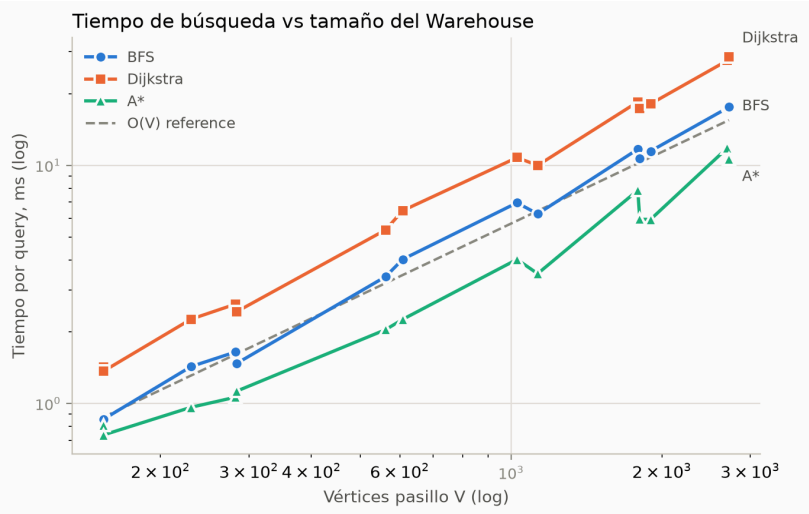


Figura C1: relación entre el tiempo de búsqueda por algoritmo y el tamaño de los warehouse de prueba en una escala log-log.

7. **¿Cómo determinaron la distancia entre las ubicaciones de una wave?**

Centroide	Spectral Clustering
Mediante BFS, Dijkstra y A*	La forma de medir distancias se realizó de dos formas distintas seleccionables antes de la ejecución. Por un lado se aplicó una distancia euclidiana (linea recta). Y por el otro una distancia topológica a partir de BFS

8. **¿Cómo decidieron qué pedidos pertenecen a una misma wave?**

Centroide	Spectral Clustering
Se ponderan todos los pedidos por prioridad y por la distancia de los productos del pedido con distancia Manhattan normalizada y se le calcula el centroide del pedido. Luego se evalúa la capacidad del picker y se toma un pedido mejor ponderado y se evalúa si a lo sumo puede cargar un producto del mismo, luego se evalúa si puede cargar el pedido completo o una fracción. En caso de que le quede capacidad al picker busca los pedidos más cercanos al mismo comparando el centroide con el resto de pedidos restantes y corrobora devuelta aquellos pedidos completo o parciales que puede guardar hasta que llegue al límite de las restricciones de la wave o no haya más capacidad de carga. Vuelve a arrancar este ciclo hasta que no hayan mas pedidos	A partir de la creación de zonas con spectral clustering, se distribuyen los productos correspondientes a los pickers hasta que cada uno complete su capacidad disponible. En caso de que se complete la zona antes de que todos los pickers estén completos, se recurre a la zona adyacente y se mandan los pickers ociosos a ella. De modo que reduce drásticamente la cantidad de ticks empleados

9. **¿Qué estrategia utilizaron para ordenar las ubicaciones de una wave?**

Centroide	Spectral Clustering
Una vez que cada picker tiene su lista de pedidos correspondiente a la wave, se ordenan por cercanía a partir de un BFS que corta cuando alcanzó todos los productos a recoger. El orden encontrado será el orden en el que el picker recoja los produtos	Una vez que cada picker tiene su lista de pedidos correspondiente a la wave, se ordenan por cercanía a partir de un BFS que corta cuando alcanzó todos los productos a recoger. El orden encontrado será el orden en el que el picker recoja los produtos

10. **¿Qué diferencias existen entre las estrategias implementadas?**

La diferencia entre el método del centroide y el Spectral Clustering radica en la optimización, ya que al separar el warehouse en partes más pequeñas las wave que se forman no dependen del pedido sino de la cercanía de los productos en dicha zona. Por lo que al generar wave, los picker recogen productos de distintos pedidos que se encuentran muy cerca entre sí, lo que optimiza los recorridos y tiempos notablemente. Mientras que con el centroide, si los productos se encuentran alejados al buscar el próximo pedido más cercano con respecto al centroide puede que con distancia euclidiana sea cercana entre pedidos, pero el producto se puede encontrar alejado con respecto al centroide provocando mayores recorridos y por ende más tiempo de ejecución.

11. ¿Cuál presenta menor distancia recorrida?

Spectral Clustering

12. ¿Cuál presenta menor tiempo de ejecución?

Spectral Clustering

13. ¿Existe una estrategia que sea mejor en todos los escenarios?

En la mayoría de los escenarios Spectral Clustering supera a la del centroide, a excepción de los casos de warehouses pequeños que suelen arrojar resultados mejores con centroide ya que la lógica de asignación de pickers está pensada para cantidades más grandes de productos

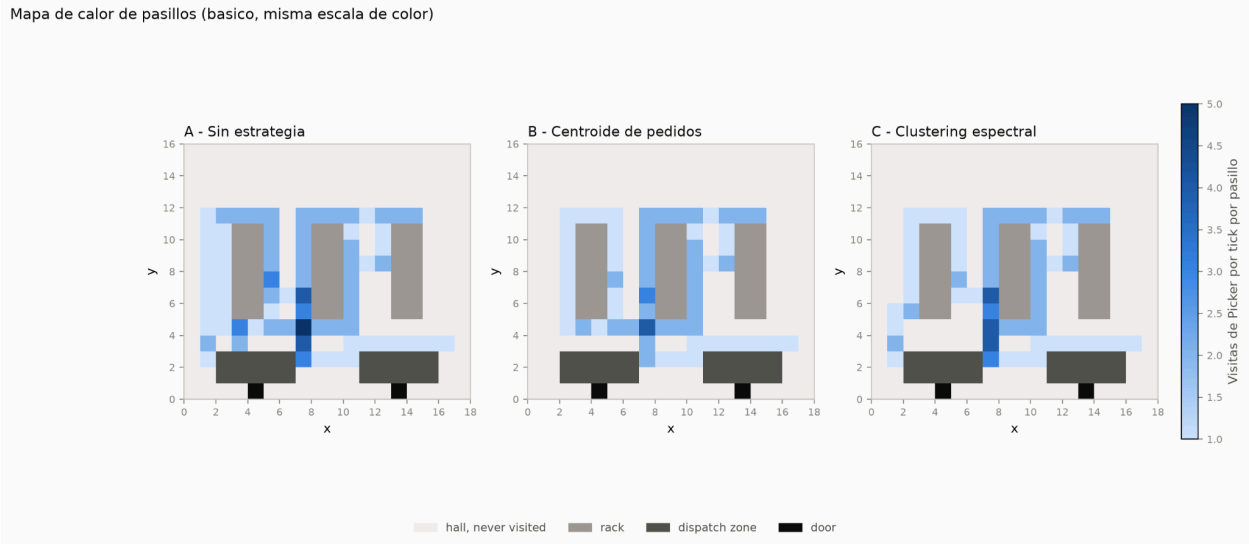
14. ¿Qué trade-offs encontraron entre tiempo de ejecución y calidad del recorrido?

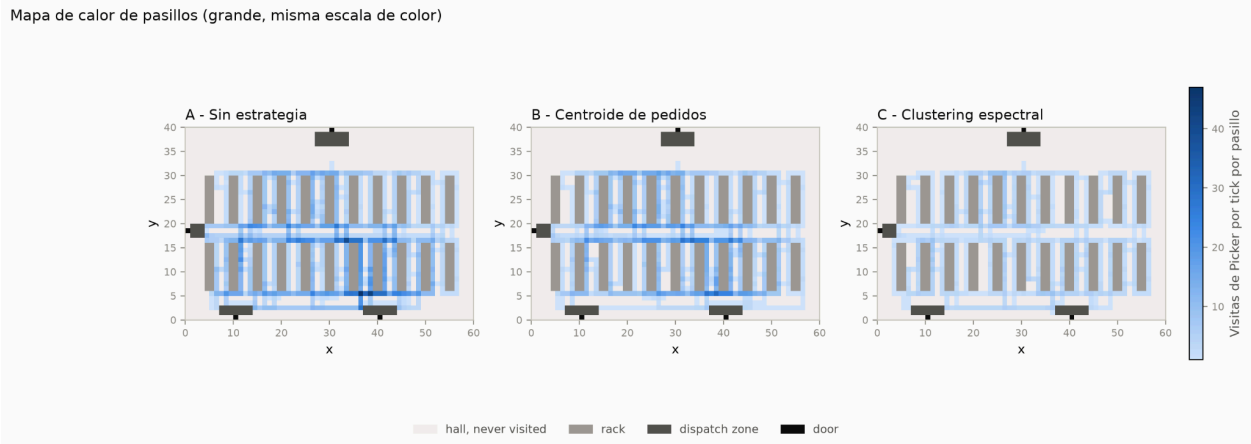
15. ¿Qué modificaciones realizarían para utilizar el sistema en un Warehouse real?

Se investigaron otros algoritmos como OR-OPT, clustering jerárquico aglomerativo que no se llegaron a implementar como soluciones alternativas debido a la falta de tiempo.

Por otro lado, si se empiezan a considerar otras heurísticas y modelos de machine learning el alcance de soluciones se amplía enormemente pudiendo aplicar algoritmos iterativos de IA para encontrar una mejor solución.

Visualización de diferencias de recorridos entre estrategias





Mapa de calor del recorrido de los pickers en los escenarios dados, por cada estrategia implementada, en una escala monocromática.